

Getting started with AWS Lambda

AWS Prescriptive Guidance

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume — there is no charge when your code is not running. This guide covers Lambda fundamentals, execution models, triggers, concurrency, performance tuning, security, and best practices for building reliable, scalable, and cost-effective serverless applications.

Authors: Fatima Al-Hassan and David Nguyen, Amazon Web Services (AWS)

Date: March 2026

Table of Contents

- [Introduction](#)
- [How Lambda Works](#)
- [Supported Runtimes](#)
- [Event Sources and Triggers](#)
- [Concurrency and Scaling](#)
- [Lambda Layers and Extensions](#)
- [Networking](#)
- [Security](#)
- [Performance and Cost Optimization](#)
- [Observability](#)
- [Best Practices](#)
- [FAQ](#)

Introduction

AWS Lambda is a serverless compute service that runs your code in response to events and automatically manages the underlying compute resources. You can use Lambda to run code for virtually any type of application or backend service. Lambda runs your code on high-availability compute infrastructure and performs all of the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, code monitoring and logging.

Lambda functions can be triggered by over 200 AWS services and SaaS applications. Common integration patterns include processing S3 events, responding to API Gateway requests, consuming messages from SQS and Kinesis, transforming data in DynamoDB Streams, and running scheduled tasks with EventBridge.

Note: Lambda charges are based on the number of requests and the duration of code execution, measured in 1-millisecond increments. The Lambda free tier includes 1 million free requests and 400,000 GB-seconds of compute time per month.

How Lambda Works

When a Lambda function is invoked, Lambda allocates a compute instance from a pool of available instances in the same AWS Region. This process — called a cold start — involves downloading the function code, initializing the runtime, and executing the initialization code in the function handler. Subsequent invocations may reuse the same execution environment, reducing latency through warm starts.

Lambda execution environments are sandboxed using micro-virtual machines (MicroVMs) powered by Firecracker, an open-source virtualization technology developed by AWS. Each execution environment has its own isolated CPU, memory, file system (/tmp), and network stack.

Lambda Invocation Modes

Lambda supports three invocation modes. Synchronous invocation waits for the function to complete and returns the result — used by API Gateway, ALB, and Lambda URLs. Asynchronous invocation places the event in a queue; Lambda retries failed executions up to two additional times — used by S3, SNS, and EventBridge. Event source mapping polls streams and queues and invokes Lambda in batches — used by SQS, Kinesis, DynamoDB Streams, and MSK.

Supported Runtimes

Lambda supports multiple programming language runtimes. AWS provides managed runtimes for Node.js, Python, Java, Go, Ruby, .NET, and custom runtimes via the Lambda Runtime API. You can also package

your function as a container image (up to 10 GB) using any base image, providing maximum flexibility for complex dependency requirements.

Runtime	Versions Supported	Init Time	Best For
Node.js	18.x, 20.x	Fast	API backends, lightweight event processing
Python	3.10, 3.11, 3.12	Fast	Data processing, ML inference, automation
Java	8, 11, 17, 21	Moderate–Slow	Enterprise apps, microservices
Go	1.x	Very Fast	High-performance, low-latency workloads
.NET	6, 8	Moderate	Windows workloads migrated to Lambda
Ruby	3.2	Moderate	Web backends, scripting
Container Image	Any language	Varies	Large dependencies, existing Dockerfiles

Event Sources and Triggers

Lambda functions can be triggered by a wide variety of AWS services and external event sources. Understanding the invocation type and batching behavior of each event source is essential for designing reliable and cost-effective Lambda-based architectures.

Amazon API Gateway and Lambda Function URLs provide HTTP/HTTPS endpoints for synchronous Lambda invocations, enabling serverless REST and HTTP APIs. Amazon SQS triggers Lambda functions when messages arrive in a queue, with configurable batch sizes and batch windows. Amazon Kinesis Data Streams and DynamoDB Streams trigger Lambda in near-real-time as new records are published.

Amazon EventBridge (formerly CloudWatch Events) can trigger Lambda on a schedule (cron or rate expressions) or in response to events from AWS services, SaaS partners, or custom event buses. Amazon S3 triggers Lambda when objects are created, modified, or deleted.

Concurrency and Scaling

Lambda automatically scales by running more execution environments in response to increased invocations. The maximum number of concurrent executions for a function is controlled by a concurrency limit. By default, all functions in a Region share a pool of 1,000 concurrent executions, though this can be increased via a quota increase request.

Reserved Concurrency

Reserved concurrency guarantees a maximum number of concurrent executions for a specific function, preventing it from consuming capacity needed by other functions in your account. Reserved concurrency can also be set to zero to effectively throttle a function, which is useful for maintenance windows.

Provisioned Concurrency

Provisioned Concurrency initializes a specified number of execution environments before invocations occur, eliminating cold start latency. It is ideal for latency-sensitive applications such as user-facing APIs. Provisioned Concurrency is charged separately from standard Lambda pricing and can be configured with Application Auto Scaling to scale based on a schedule or CloudWatch metrics.

Lambda Layers and Extensions

Lambda Layers are ZIP archives that contain libraries, a custom runtime, or other function dependencies. Layers are extracted to `/opt` in the execution environment. You can include up to five layers per function. Layers allow you to manage shared dependencies centrally, reducing package sizes and deployment times.

Lambda Extensions integrate monitoring, observability, security, and governance tools into the Lambda execution lifecycle. Extensions can run in-process (as part of the runtime process) or as external processes alongside the runtime. AWS Partners such as Datadog, New Relic, and Dynatrace offer Lambda extensions for integrated observability.

Networking

By default, Lambda functions can access the internet and AWS services. If your function needs to access resources in a VPC — such as an RDS database or ElastiCache cluster — you can configure it to connect to the VPC. Lambda creates Elastic Network Interfaces (ENIs) in your VPC using Hyperplane, which enables highly scalable VPC connectivity without the ENI scaling limits of earlier Lambda VPC implementations.

Note: Functions connected to a VPC do not have outbound internet access by default. To enable internet access, route outbound traffic through a NAT Gateway in a public subnet.

Performance and Cost Optimization

Lambda performance is primarily controlled by the configured memory setting (128 MB to 10,240 MB). CPU allocation scales proportionally with memory — at 1,769 MB, a function has the equivalent of one full vCPU. Allocating more memory can reduce execution duration, which may reduce overall cost even though the per-GB-second rate is the same.

AWS Lambda Power Tuning is an open-source tool that runs your function with different memory configurations and measures execution time and cost, helping you find the optimal memory setting. ARM/Graviton2-based Lambda functions (available by selecting the arm64 architecture) can deliver up to 34% better price-performance compared to x86 for many workloads.

Best Practices

- Keep functions small and focused: Each Lambda function should do one thing well, following the single responsibility principle.
- Minimize cold start impact: Use Provisioned Concurrency for latency-sensitive functions, choose fast-starting runtimes (Node.js, Python, Go), and minimize deployment package size.
- Use environment variables for configuration: Never hardcode configuration; use environment variables for runtime configuration and AWS Secrets Manager for sensitive values.
- Design for idempotency: Since Lambda functions may be retried, design them to produce the same result when invoked multiple times with the same event.
- Reuse execution context: Initialize SDK clients, database connections, and other heavy objects outside the handler function to reuse them across warm invocations.
- Set appropriate timeouts: Choose a timeout that is long enough for your function to complete under normal conditions, but short enough to fail fast under abnormal conditions.
- Use Dead Letter Queues (DLQs): Configure DLQs for asynchronous invocations to capture failed events for debugging and reprocessing.
- Enable X-Ray tracing: AWS X-Ray provides end-to-end distributed tracing for Lambda functions, helping you identify performance bottlenecks and errors.

Frequently Asked Questions

Q: What is the maximum execution duration for a Lambda function?

A: The maximum execution timeout is 15 minutes (900 seconds). For workloads requiring longer execution, consider AWS Fargate, EC2, or AWS Step Functions to orchestrate multiple Lambda functions.

Q: How can I reduce cold start latency?

A: Use Provisioned Concurrency, choose a fast runtime (Node.js or Python), minimize package size, avoid VPC configurations if not needed, and use Lambda SnapStart for Java functions.

Q: Can Lambda access resources in a VPC?

A: Yes. You can configure a Lambda function to connect to your VPC by specifying the VPC, subnets, and security groups. Lambda uses Hyperplane for scalable VPC connectivity.